🌳 1. What is a Tree?

A **tree** is a non-linear data structure that simulates a hierarchical tree structure with a **root node** and **sub-nodes (children)**.

- Consists of **nodes** connected by **edges**
- Has a **single root node**
- Nodes with no children are **leaf nodes**
- Can be **recursive** in nature

🧠 2. Key Terminology

Term	Description
Root	Top node of the tree
Parent	A node that has children
Child	A node derived from a parent
Leaf	Node with no children
Siblings	Nodes with the same parent
Depth	Number of edges from root to node
Height	Max depth of any node from the root
Subtree	A tree formed from a node and its descendants
Binary Tree	A tree where each node has max two children

📂 3. Types of Trees

Type	Description
Binary Tree	Each node has ≤ 2 children
Binary Search Tree (BST)	Left < Root < Right
Balanced Binary Tree	Height difference ≤ 1 for left & right subtrees
AVL Tree	Self-balancing BST with rotations

Type	Description
Red-Black Tree	Balanced BST using color properties
N-ary Tree	A node can have N children
Heap Tree	Complete binary tree (Min-Heap / Max-Heap)
Trie	Prefix tree used for string searching
Segment Tree	Used in range queries
B-Tree / B+ Tree	Self-balancing tree used in databases and file systems

---

🔍 4. Applications of Trees

- **Binary Search Tree**: Fast search, insert, delete (O(log n) average)
- **Heaps**: Priority Queues (used in Dijkstra's algorithm)
- **Tries**: Auto-complete, spell-checker
- **Segment Tree**: Range queries (sum/min/max in intervals)
- **Syntax Tree**: Compiler design
- **File System Hierarchy**
- **Routing Tables in Networks**

🧰 5. Basic Tree Implementation in Python

```
class Node:
    def __init__(self, value):
        self.data = value
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, root, value):
        if root is None:
            return Node(value)
        if value < root.data:
            root.left = self.insert(root.left, value)
        else:
            root.right = self.insert(root.right, value)
        return root
```

```
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=' ')
            self.inorder(root.right)

# Example usage
tree = BinaryTree()
values = [10, 5, 15, 2, 7, 12, 20]
for v in values:
    tree.root = tree.insert(tree.root, v)

print("Inorder Traversal:")
tree.inorder(tree.root)
```

🎯 6. Traversal Methods

Method	Order
Inorder	Left → Root → Right
Preorder	Root → Left → Right
Postorder	Left → Right → Root
Level Order	Breadth-first traversal

Great! Let's now focus on **BFS** and **DFS in the context of Trees**.

🔍 DFS (Depth-First Search) in Trees

DFS explores **deep into the tree** before backtracking. In trees, this includes **3 classic traversal types**:

✅ 1. Preorder Traversal

- Visit **root**, then **left**, then **right**
- Order: `Root → Left → Right`

✅ 2. Inorder Traversal

- Visit **left**, then **root**, then **right**
- Used mainly in **Binary Search Trees** (BST) to get sorted order

- Order: `Left → Root → Right`

✅ 3. Postorder Traversal

- Visit **left**, then **right**, then **root**
- Useful for **deleting tree**, **bottom-up evaluations**
- Order: `Left → Right → Root`

🧪 Python Code for DFS (Preorder, Inorder, Postorder)

```
class Node:
    def __init__(self, value):
        self.data = value
        self.left = None
        self.right = None

def preorder(node):
    if node:
        print(node.data, end=' ')
        preorder(node.left)
        preorder(node.right)

def inorder(node):
    if node:
        inorder(node.left)
        print(node.data, end=' ')
        inorder(node.right)

def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=' ')

# Example tree
'''
       1
      / \
     2   3
    / \
   4   5
'''

root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)

print("Preorder:"); preorder(root)
```

```
print("\nInorder:"); inorder(root)
print("\nPostorder:"); postorder(root)
```

🔁 BFS (Breadth-First Search) in Trees

Also known as **Level Order Traversal**. Visits nodes **level by level** from top to bottom.

🧠 Logic:

- Use a **Queue**
- Push root → while queue is not empty, pop and print → enqueue left and right child

🧪 Python Code for BFS (Level Order)

```
from collections import deque

def bfs(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.data, end=' ')
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

print("\nLevel Order Traversal (BFS):")
bfs(root)
```

📊 DFS vs BFS in Trees

Feature	DFS (Pre/In/Post)	BFS (Level Order)
Traversal Type	Depth-first	Breadth-first (top-down)
Structure Used	Recursion or Stack	Queue
Memory Use	Depends on tree height	Depends on max width
When to Use	Tree evaluation, expression tree	Shortest path, layer-wise ops

---

✅ Use Cases in Trees

- ✅ **DFS (Preorder)**: Tree cloning, prefix notation
- ✅ **DFS (Postorder)**: Deletion, postfix eval
- ✅ **DFS (Inorder)**: BST sorting
- ✅ **BFS**: Level order printing, shortest path in unweighted tree

---